

Our video, for example, has two scenes—the first one with a shot of the car revving up and jumping off the table; the second is the fall itself. It is this fall that we freeze in time, pan around, and then continue.

The Basics Of Bullet Time

So how do these bullet time videos work, anyway? The effect is simple enough (well, on paper, at least) to create:

- ❑ Take large number of cameras, arrange in circular arc
 - ❑ Wait for subject to enter this arc
 - ❑ Freeze video feed from first camera
 - ❑ Take still picture from all but the last camera at this instant
 - ❑ Show each of these stills in the video to convey the illusion that time has stopped and the camera is moving around the subject
 - ❑ Continue video from last camera
- Now all we have to do is reproduce this.

Ready... Action!

Our setup follows these guidelines, if a little liberally. The location: an abandoned meeting room in our building. As you can see in the picture, we've got eight cameras arranged in an arc on the floor to capture the car in the middle of its fall. You also need to make sure that all the cameras are focused on the same spot. To do this, place an object at the point where you expect the action to happen—where the car hit the ground, in our case—and focus each camera on this point.

Ideally, you should have identical cameras between the first and the last. This way, it becomes easier for you to take a still picture in the middle of the action and have a chance at capturing the same moment on all cameras, thanks to their identical response times. Realistically, though, you'll be dealing with a bunch of different cameras with different response times between clicking the button and the actual taking of the photo, so video mode for all of them would be advisable—we'll just extract the frame we want from the videos. Make sure all of them are recording video at the same resolution—320 x 240 is a likely common denominator.

Ideally, you should have identical cameras between the first and the last

As with many things, more is better—at 25 frames per second, a one-second bullet time effect requires 25 cameras (since we're showing one frame from each camera). High-budget movies use *at least* a hundred. So don't be too picky when soliciting partners in this crime!

Once you've got your cameras and scene set up, there's nothing left to do but start filming! Since we've chosen video mode for all cameras, we don't need to worry about recording at the exact same moment. The trade-off is that we'll have considerable motion blur to deal with, depending on how fast your subject is moving. Canon cameras come with a "Fast Frame Rate" setting that counters this a bit. The cameras otherwise used for this kind of effect are extremely quick, recording more than a hundred frames per second as opposed to the basic camera's thirty.

Turn on all the cameras (extra hands are a big help here) and enact your scene—this completes the easiest part of this exercise.

The Preliminaries

Once you've finished recording and copied all the video feeds to your PC (a good idea is to name the feeds camera01, camera02 and so on), the first thing you need to do is extract all the frames from the movies. You'll require plenty of hard drive space for this. Start VirtualDub and open the first movie. Go to File > Save Image Sequence and select your options from here. To save time, check the box at the bottom to add this to VirtualDub's job queue. Repeat this for all the video feeds, and when you're done, hit [F4] to open the Job Control dialog, click Start to begin extracting all the frames and go get yourself a snack—the process consumes some time.

Once this is done, we begin the tedious process of identifying the perfect frame for the effect—ideally one with as little motion blur as possible—from each of the cameras. Once you've isolated these frames, rename them for easy reference.

Bring It All Together

Start Windows Movie Maker (WMM—it's bundled with Windows XP under Start > Programs) and



A view of the entire scene we set up

Focus all the cameras on one point

Extract all the frames in VirtualDub